Find your next words.

Learn a language through the conversations you are already having.

Google AI | Cloud Run | Web + Android

Suggested social-impact category: **Education**
Theme alignment and eligibility to be confirmed with the event.

ONE CONVERSATION, A SMALL LESSON

Hear the words.
Understand them.
Find a reply.

Tap a word. Learn its meaning.

> Web v1.4 is live on Cloud Run: Gemini 3.8 Flash, one-call lessons, minimal UI and first-use tutorial. Hosted Google transcription, readings and contextual replies are verified.

Help at the moment a conversation happens.

Learners need to understand what they heard and know what they can say next—at a café, at work or while travelling.

01 Choose two languages

A first-use tutorial helps select your languages. Connect with a host code or your own Google key.

02 Press the big speaking button

See finalized speech, a translation and three suggested replies. Typed phrases work too.

03 Tap, practise and revisit

Tap words for a meaning bubble, switch to pronunciation, hear or copy replies, and reopen saved learning.

> Intended benefit: make everyday language practice more accessible through real context. Learning outcomes have not yet been measured.



Actual local web UI · prepared Japanese example
“Would you like some coffee?” · illustrative, not a new live AI result

SYSTEM ARCHITECTURE

Speech becomes a lesson through Google AI.

One HTTPS origin serves the website, authenticated audio relay and lesson API.

ON THE LEARNER'S DEVICE

Browser

Microphone → AudioWorklet → mono 16 kHz PCM audio.

Interim captions stay temporary. Final phrases enter the learning queue.

Lessons and vocabulary are saved locally.

→

GOOGLE CLOUD RUN

FastAPI relay

Authenticate first, then enforce request bounds, pacing, quotas and concurrent-call limits.

WebSocket carries audio; HTTP carries the text, context and optional recent sound.

Provider credentials stay server-side.

⇄

GOOGLE AI MODELS

Transcribe + Flash

Gemini 3.5 Transcribe Live
Produces speech transcription across languages.

Gemini 3.8 Flash · v1.4
One low-reasoning call returns translation, replies, word meanings and pronunciation.

Google response → validation: preserve the exact source text; require word definitions and three replies for a learning-language lesson.

Browser: render tappable words, translations, pronunciation and saved history.

Model IDs are server-configured. Web 1.4 uses explicit required word fields and bounded readings; Latin words reuse their original script. Transcribe Live is a separate service.

Relevant source: backend/app.py · web/microphone.js · web/app.js · shared/lesson_schema.json

CONTEXT AND RESPONSE GENERATION

Replies follow what is happening now.

A goal is optional. The same Flash request can infer the immediate situation and suggest ways to respond.

BOUNDED CONTEXT IN EACH LESSON REQUEST

1. Current utterance and selected language pair
2. Up to eight earlier conversation turns
3. A scene summary and the previous three suggestions
4. Editable learner notes and up to twelve seen words
5. An explicit goal or tentative inferred intent

An explicit goal overrides a guess. New conversation clears the scene sent to Google while retaining Saved.

ILLUSTRATIVE SCENARIO

Learner’s goal

“I want vegetarian takeaway.”

A later question

“Will you eat here?”

A context-aware reply could be:
“To take away, please.”

Illustrative scenario. With a blank goal, the model uses the immediate exchange; conversational accuracy needs broader user testing.

Suggestions are kept separate from spoken facts. “Likely you” is a conversational estimate, not verified speaker identity. The learner can confirm “I said this.”

No extra summarization or speaker-classification model call

MEMORY AND PRIVACY

Learning stays local. Live signals stay temporary.

INFORMATION	STORAGE AND USE	LIFETIME / CONTROL
Saved phrases and lessons	IndexedDB in the browser; searchable words, replies and meanings	Reopen or delete in Saved; clearing site data removes them
Learner profile and notes	LocalStorage; separate learning namespaces on this browser	Notes are editable; profiles are not separate authenticated accounts
Conversation scene	Local conversation state; bounded earlier text sent with a lesson	History over 90 minutes old is excluded; New conversation resets the scene
Optional location	RAM only; coordinates rounded to two decimal places	Starts off; location expires after five minutes; no Maps API request
Optional recent sound	RAM only; up to four seconds of WAV sent alongside transcription to Flash	Starts off; no raw audio is saved in lesson history
API key and session cookie	Key in server RAM; cookie contains a signed opaque session identifier	Up to one hour; disconnect, expiry or server restart removes key-session access

HTTPS, HttpOnly and SameSite=Strict protect browser sessions. Cookie-authenticated writes and WebSocket connections require the app’s own origin.

The server has no transcript/history database. Users must trust the host receiving their key; anyone using the same browser can switch local learner profiles.

Use a host code or bring your own key.

USE ACCESS CODE

Host-funded Google calls

The private code creates a signed browser session. The recommended Cloud Run setup calls Vertex AI using the runtime service account.

Google model usage: billed to the host's Cloud project.

USE MY GOOGLE API KEY

Learner-funded Google calls

A short server-memory session uses that Google AI Studio key for both transcription and Flash. Failed keys never fall back to host credentials.

Google model usage: billed to the personal key's project.

1 instance
Scale to zero when idle
512 MiB · one CPU

2 + 2
Two active listeners
Two concurrent lessons

24 / minute
Lesson starts
360/hour · 400/day

40 / 60 min
Combined audio allowance
per hour / per day

Host-code and personal-key users have separate shared model allowances; all personal-key sessions share theirs. Both groups share infrastructure concurrency and pacing limits. The host still pays Cloud Run relay costs.

> Quotas are per process and reset after a restart. They are not a hard billing cap. A global AI pause stops paid routes. Scale-to-zero also forgets personal keys, so users may need to reconnect.

A functional prototype with clear next steps.

CURRENT-SOURCE BACKEND

80

Checks passed

Authentication, quotas, structured lessons, PCM validation, context bounds, provider/key isolation, session expiry and revocation.

Upstream Google HTTP and WebSocket behaviour is mocked in these tests.

CURRENT-SOURCE BROWSER

57

Checks passed

Real Chromium UI, cookies, IndexedDB, microphone worklet, word bubbles, memory, sensors and both connection modes.

Includes first-use tutorial checks. Synthetic microphone/location; Google responses are mocked.

CURRENT LIVE DEPLOYMENT

Cloud Run + Google AI

Web v1.4 passed real Google transcription and a Gemini 3.8 context lesson using a 2.128-second synthesized-speech fixture plus the optional WAV input.

One context/audio sample took 14.2 seconds; not a latency benchmark. Valid personal-key inference and physical-phone capture remain unverified.

Feasible demo today: a framework-free web client and one FastAPI service reuse the existing Google model APIs. There is an Android companion; this PDF focuses on the web flow.

For a broader launch: durable per-user quotas, stronger user identity, a design for cross-instance sessions, device/accessibility testing and measured learning outcomes. These are future work.

Evidence and limitations: VALIDATION.md · validation/web-*-results.json

Explore the app and its implementation.

Live Cloud Run prototype

lingolive-358650584819.us-central1.run.app

Web v1.4: use the host’s private code or your own Google AI Studio key. No credentials are included here.

Independent source repository

github.com/U4AR/LingoLive

Repository was private at publication. Judges need repository access, or the owner must change visibility.

Technical explanation and evidence

Memory and session design

Validation and deployment status

Status at publication · 17 October 2026

Before submitting

1. Try the live v1.4 flow on representative phones and languages; assess latency and learning usefulness.
2. Give judges app and repository access; verify the latest live flows.
3. Record a public three-minute video: listen, see replies, tap a word and revisit Saved.
4. Confirm the eligible problem statement, category, deadline and remaining submission rules.

This English PDF provides the project explanation and proposed impact. No demo video or completed hackathon submission is claimed.